

AKASH RAJPURIA

MLOPS

<https://www.linkedin.com/in/akashrajpuria/>

① Doubts

② Notes

③ Agenda

↳ ML

↳ Git → Dev

↳ Streamlit

↳ API

Docker

ECK / ECS / Kuber

CI CD

MLflow / Optuna

} Intro

Simple Experience-Salary Dataset

	Experience (Years)	Salary
1	1	10k
2	2	20k
3	3	30k
4	4	40k
5	5	50k
	6	60k

Machine Learn

$$\begin{aligned} \text{Exp} \times \text{Salary} &= \text{Sal.} \\ 6 \times 10k &= 60k \end{aligned}$$

Model

$$4 \times 30k = 120k$$

50k

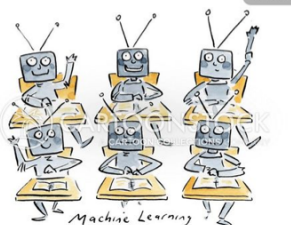
1 - ML Model

Model.py

```
from sklearn.linear_model import LinearRegression
# Create and train the model
model = LinearRegression()
model.fit(experience, salary)
# Predictions
predicted_salary = model.predict(experience)
```

Learn

60k



DS / ML

Reward
Loss
Score
ML

Model

Available for complete world

MLOPS operation

2 - What is Version Control?

Version Control is like a time machine for your code and models. It tracks every change, who made it, and when.

- See what changed
- Who changed it
- When it was changed
- Go back to any previous version



Why We Need Version Control?

Real Scenario: Your team is building a UPI fraud detection model:

- Rahul improves accuracy to 95%
- Priya accidentally breaks the code
- Without version control: 🚨 All work lost!
- With version control: 😊 Revert to Rahul's version in seconds

Franky
Png

git set go run

```
# Initialize Git repository for your Aadhaar verification project
git init aadhaar-verification-m1

# Track your first model
git add aadhaar_model.pkl
git commit -m "Initial Aadhaar face verification model - 85% accuracy"

# Priya improves the model
git add aadhaar_model.py
git commit -m "Added age detection feature - accuracy now 89%"

# Rahul adds fraud detection
git add fraud_detection.py
git commit -m "Added synthetic Aadhaar detection - catches 95% fake IDs"

# View history
git log --line
Output:
1a4f585 Added synthetic Aadhaar detection - catches 95% fake IDs
2b3c4d5 Added age detection feature - accuracy now 89%
1a2b3c4 Initial Aadhaar face verification model - 85% accuracy

Someone broke the code? No problem!
git revert 1a4f585 # Go back to Priya's stable version

# Working on Diwali discount feature? Create a branch
git checkout -b diwali-special-verification
# Make changes without affecting main code
```

→ create empty repo git

adhd file

commit

push

Git init → Start a project

Git add + commit → Save milestones

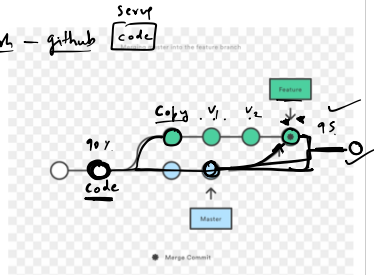
Git log → See project history

Git revert → Go back to stable versions

Git branch → Experiment without breaking main code

push → github

code



What is DVC?

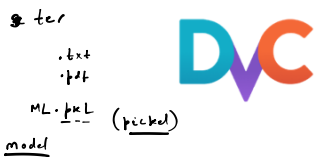
DVC (Data Version Control) = Git for data & ML models.

- Git is great for **code** (small text files). **code**
- But **ML projects** also have **data (GBs)** and **models (MBs/GBs)** → Git can't handle them. **ML 2GB**
- DVC helps you **version, share, and reproduce** data + models, just like Git versions your code.

Where is DVC used in a pipeline?

Imagine an ML pipeline:

- Data Collection** → CSV, images, logs, etc.
DVC stores versions of these big datasets.
e.g. `for file in train_images.csv`
- Data Cleaning / Feature Engineering** → Store intermediate processed datasets.
e.g. `for file in datacleaned_transactions.csv`
- Model Training** → Store the trained model files.
e.g. `for file in modeltrained_model.pkl`
- Evaluation / Deployment** → Track which dataset + code produced which model.



```
# track model_versions.py
import dvc.api
import os

# Version control for large model files
def save_model_version(model, metrics, description):
    """
    Save model with version control
    Like saving different versions of Dosa recipes!
    """
    # Save model file
    model_path = f"models/api_fraud_model_{metrics['accuracy']}.pkl"
    os.system(f"cp {model} {model_path}")

    # Track with DVC (handles large files)
    os.system(f"dvc add {model_path}")
    os.system(f"dvc commit -m '{description}'")

    # Push to remote storage (like S3)
    os.system("dvc push")

    return model_path
```

SS, file.

gauri .pkl

Example usage

save model version

- Git = **recipe book (code)**
- DVC = **fridge (stores actual dosa batter & chutney = data & models)**
- Together, you can always reproduce the same dosa

voice Jio Kiyan

webkit {HTML CSS} Demo

What is Streamlit?

Streamlit is like PowerPoint for ML models

It turns Python scripts into beautiful web apps in minutes. No HTML/CSS/JavaScript needed!

Run a Streamlit app from a script

Model Development → Streamlit Demo → Stakeholder Approval → Production Deployment

Quick Feedback Loop

über-Akt - gM

Simple Analogy:

-
- The diagram illustrates the architecture of a mobile application. It features three main components: a **Frontend** (represented by a smartphone icon), a **Backend** (represented by a server icon), and **Google APIs** (represented by a robot icon). The **Frontend** is labeled with 'Frontend' above and 'Frontend' below. The **Backend** is labeled with 'Backend' above. The **Google APIs** are labeled with 'Google APIs' above and 'Google APIs' below. Arrows indicate the flow of data: an arrow labeled 'API CALL' goes from the **Frontend** to the **Backend**; an arrow labeled 'API CALL' goes from the **Backend** to the **Google APIs**; and an arrow labeled 'API CALL' goes from the **Google APIs** back to the **Frontend**. A small icon of a person is shown in the top right corner of the diagram.

Request: "Is this UPI transaction fraud?"
Response: "Yes, 87% probability" X

```
python
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)

# Load a pre-trained model (saved earlier with joblib)
model = joblib.load("salary_model.pkl")

# Define a prediction endpoint
@app.route('/predict', methods=['POST'])
def predict_salary():
    """
    Predict salary based on experience.
    """
    # Parse the request body
    exp = request.get_json()
    exp = data["experience"]

    # Make prediction
    prediction = model.predict([exp])[0]

    # Return as JSON
    return jsonify({"experience": exp, "predicted_salary": int(prediction)})

if __name__ == "__main__":
    app.run(debug=True)
```

Copy Edit

$$\frac{I_k L}{\downarrow} \text{ free } \}$$

$$\left\{ \begin{array}{l} \text{flask} \\ \text{instapi} \end{array} \right\} \text{api}$$

~~Dix~~

7 evidence

Input $\rightarrow$ ML $\rightarrow$ predict $\rightarrow$ ML $\rightarrow$ Return

Story - Math - code
work

Develop
Tandori
corn

Test

✓ ✓ ✓ ✓ ✓ ✓

✓ ✓

flip kart

-
- A stylized illustration of a person with dark hair, wearing an orange shirt and blue pants, sitting on a white stool and working on a blue laptop. Above the person, there are several icons: a clock, a checkmark in a square, and a cloud. To the right of the person, there is a large, light blue rectangular shape representing a software window or interface, with a rocket ship launching from it, emitting a trail of orange and yellow flames. Below the rocket ship, there are three blue cylindrical shapes representing data storage or databases.

```
bash
ml-salary-app/
├── app.py           # Flask API
├── salary_model.pkl # Pretrained model (saved earlier)
├── requirements.txt # Python dependencies
└── Dockerfile       # Docker instructions
```

txt

flask
scikit-learn
joblib

3 Dockerfile

- ```

dockerfile

Use Python base image
FROM python:3.9

Set work directory
WORKDIR /app

Copy requirements.txt
COPY requirements.txt

Copy app.py
COPY app.py

Copy safety model.pkl
COPY safety_model.pkl

Install dependencies
RUN pip install -r requirements.txt

Run the app
CMD ["python", "app.py"]

```

remnants.txt  
} install  
sh  
→ [ ] flush  
out

#### 4 Build the Docker Image

Run this in the folder:

```
bash
docker build -t salary-api .
```

*Handwritten: Dockerfile, Dockerfile*

#### Run the Container

```
bash
docker run -p 5000:5000 salary-api
```

*Handwritten: Port, http://localhost:5000, 5000*

- Maps port 5000 inside Docker → 5000 on your machine.
- Flask API is now running inside a container.

#### Test the API

*Handwritten: 2046 X*

## What is ECR (Elastic Container Registry)?

- AWS service to store Docker images (like GitHub for your containers).
- You push your image here → servers & ECS can pull it.



## What is ECS (Elastic Container Service)?

- AWS service to run Docker containers at scale (without managing servers).
- You define a Task = which image + how to run.
- ECS pulls your image from ECR and runs it.

*Handwritten: Amazon, ECS, push, pull, 2M*

```
1. Authenticate Docker with AWS ECR
aws ecr get-login-password --region us-east-1 | \
 docker login --username AWS --password-stdin <your_aws_account_id>.dkr.ecr.us-east-1.amazonaws.com

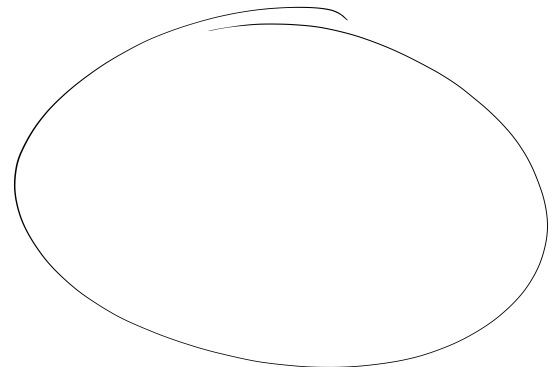
2. Create a repo in ECR (only once)
aws ecr create-repository --repository-name salary-api

3. Tag your local Docker image for ECR
docker tag salary-api:latest <your_aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/salary-api:latest

4. Push the image to ECR
docker push <your_aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/salary-api:latest
```

## ECS vs Fargate vs ECS on Fargate

- **ECS (Elastic Container Service)**
  - A service to run Docker containers on AWS.
  - But how the containers run depends on the launch type you choose.
- **ECS with EC2 launch type**
  - You manage the EC2 servers yourself (VMs where Docker runs).
  - You're responsible for patching, scaling, etc.
- **AWS Fargate**
  - A serverless compute engine for containers.
  - You don't manage EC2 instances. AWS automatically provisions resources.
  - You just say: "Run my container with 0.5 vCPU + 1GB RAM."
- **ECS on Fargate (ECS Fargate)**
  - Means you're using ECS as the orchestration tool, but Fargate as the compute backend.
  - So ECS = the brains (which tasks, how many, networking)
  - Fargate = the muscles (actually runs the containers, serverless).



```
jsonc
// task-definition.json
{
 "family": "salary-api-task",
 "networkMode": "awsvpc",
 "requiresCompatibilities": ["FARGATE"],
 "cpu": "256",
 "memory": "512",
 "containerDefinitions": [
 {
 "name": "salary-api",
 "image": "<your_aws_account_id>.dkr.ecr.us-east-1.amazonaws.com/salary-api:latest",
 "essential": true,
 "portMappings": [
 { "containerPort": 5000, "hostPort": 5000 }
]
 }
]
}
```

*Handwritten: Manager - Add delete in who need*

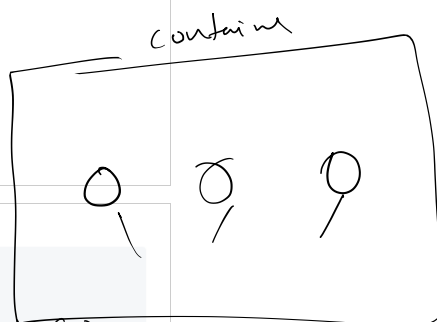
*Handwritten: ML - git - DVC - Demo - API*

ECS is the **ORCHESTRATOR** that manages your containers:

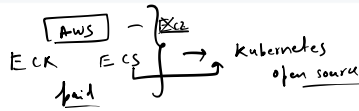
*Handwritten: Docker, ECR*

ECR (Storage) → ECR (Running) → Load Balancer → Users

- Auto-scaling ✓
- Health checks
- Rolling updates







## Where

### Kubernetes Fits in

- Docker → packages your app.
- ECS (with Fargate) → AWS-native way to run/manage Docker containers.
- Kubernetes (a.k.a. K8s) → an open-source container orchestrator that works across any cloud or on-prem.



So Kubernetes is basically a **universal orchestrator** for containers.

Kubernetes is the MASTER ORCHESTRATOR:

Code → Docker Image → Container Registry → (KUBERNETES) → Production

- Manages 1000s of containers
- Auto-healing
- Load distribution
- Multi-region deployment

```

yaml
apiVersion: apps/v1
kind: Deployment
metadata:
 name: salary-api
spec:
 replicas: 2 # run 2 copies
 selector:
 matchLabels:
 app: salary
 template:
 metadata:
 labels:
 app: salary
 spec:
 containers:
 - name: salary-api
 image: <account_id>.dkr.ecr.us-east-1.amazonaws.com/salary-api:latest
 ports:
 - containerPort: 5000

apiVersion: v1
kind: Service
metadata:
 name: salary-service
spec:
 type: LoadBalancer

```

## What is CI/CD?

CI/CD = Continuous Integration + Continuous Deployment/Delivery

- **CI (Continuous Integration)**
  - Every time a developer pushes code (e.g., new model, bug fix), the system **automatically builds, tests, and checks it**.
  - Ensures code is always working and can integrate smoothly with others.
- **CD (Continuous Deployment/Delivery)**
  - After CI, the system **automatically deploys the app/model to production** (or staging).
  - No manual "copy files to server" → automation does it.

### 1. Developer pushes code/model

```

bash
git push origin main

```

### 2. CI (Build + Test)

- GitHub Actions / Jenkins runs automatically:
  - Install dependencies
  - Run tests (unit tests, model accuracy checks)
  - Build Docker image

```

yaml
Example GitHub Action CI snippet
jobs:
 build-and-test:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2 # get code
 - run: pip install -r requirements.txt
 - run: pytest --test # run tests
 - run: docker build -t salary-api .

```

## Optima

Optima is an open-source automatic hyperparameter optimization framework for machine learning. In simpler terms, it's a tool that helps you find the best values for your model's parameters (like learning rate, number of layers, etc.) automatically, instead of tuning them manually.

It's often used in MLOps pipelines to automate and optimize model training as part of the experimentation process.

### Why It's Used in MLOps

In **MLOps (Machine Learning Operations)**, automation and reproducibility are key. Optuna helps with:

- **Automated hyperparameter tuning** — reduces manual trial-and-error.
- **Efficient search** — uses smart algorithms (like Bayesian optimization and TPE) to find good results quickly.
- **Integration** — works well with popular ML frameworks like PyTorch, TensorFlow, Scikit-learn, and XGBoost.
- **Logging & reproducibility** — integrates with MLflow, Kubeflow, or other orchestration tools to track experiments.

Example: Using Optuna with Scikit-learn

```
python
import optuna
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestClassifier

Load data
X, y = load_iris(return_X_y=True)

Define objective function
def objective(trial):
 n_estimators = trial.suggest_int("n_estimators", 10, 200)
 max_depth = trial.suggest_int("max_depth", 2, 32)
 clf = RandomForestClassifier(
 n_estimators=n_estimators, max_depth=max_depth, random_state=0
)
 return cross_val_score(clf, X, y, cv=5).mean()

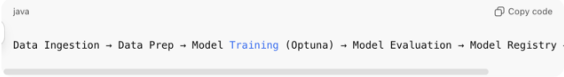
Create a study and optimize
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)

print("Best parameters:", study.best_params)
print("Best accuracy:", study.best_value)
```

How It Fits in an MLOps Workflow

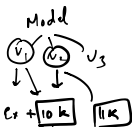
How It Fits in an MLOps Workflow

Optuna is typically part of the "Model Training" or "Experimentation" step:



It can be integrated with:

- **MLflow** → for experiment tracking
- **Kubeflow Pipelines / Airflow** → for automated training
- **Docker & CI/CD** → for reproducible MLOps pipelines



ML-FLOW

MLflow is an open-source platform to manage the end-to-end machine learning lifecycle. It helps data scientists and ML engineers track experiments, manage models, and deploy them efficiently. MLflow = version control + tracking + packaging + deployment for ML models.

Why MLflow is Important in MLOps

In MLOps (Machine Learning Operations), the goal is to make ML models **reproducible, automated, and manageable** — just like software.

MLflow provides the foundation for this by offering 4 main components:

| Component       | Purpose                                                                 |
|-----------------|-------------------------------------------------------------------------|
| MLflow Tracking | Log parameters, metrics, code versions, and artifacts from experiments. |
| MLflow Registry | Package ML code in a reusable, reproducible format.                     |
| MLflow Models   | Manage and store models in a standard format (with versions).           |
| MLflow Registry | Central hub for model versioning, approvals, and deployment tracking.   |

MLflow for Experiment Tracking

Example: Using MLflow for Experiment Tracking

Let's say you're training a logistic regression model — you can use MLflow to track your experiments automatically.

```
python
import mlflow
import sklearn
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

Load data
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

Start MLflow experiment
mlflow.set_experiment("Iris_Classification")

with mlflow.start_run():
 # Train model
 model = LogisticRegression(C=0.1, solver="libsvm")
 model.fit(X_train, y_train)
```

How MLflow Fits into MLOps

In a Real MLOps Workflow

| Stage                       | Tool                          | Role                        |
|-----------------------------|-------------------------------|-----------------------------|
| Data Versioning             | DVC / Delta Lake              | Manage data versions        |
| Experiment Tracking         | MLflow                        | Track runs, params, metrics |
| Hyperparameter Optimization | Optuna                        | Auto-tune parameters        |
| Model Registry              | MLflow Registry               | Store and version models    |
| Deployment                  | Docker / Kubernetes / FastAPI | Serve models in production  |
| Monitoring                  | Prometheus / Grafana          | Track drift & performance   |